Lab 3

Week 3

Introduction to On-Chip Communication and AMBA-AXI

AXI-Lite slave peripheral (register-mapped design).

The AXI Protocol

When building your first block diagram or reading the documentation of Xilinx's IP cores, you may notice one thing in common. They all use the AXI protocol. This Lab will provide a brief explanation about what AXI is and how it functions.

Essentially, the AXI protocol outlines the process by which on-chip components can communicate with each other using signals, usually involving a master and slave device. By standardizing this protocol, we can ensure every peripheral and IP core present on an FPGA will be able to talk to each other, creating a cohesive system (rather than a scattered collection of cores).

There are three types of AXI4 interfaces (defined by AMBA 4.0):
Full AXI4 - High-performance communication, using memory-mapped addresses (more here).
AXI-Lite - Lightweight and simple memory-mapped interface, used for single transaction communication.
AXI4-Stream - 'Direct' device communication, removing the need for addresses and allowing for maximum data transfer.

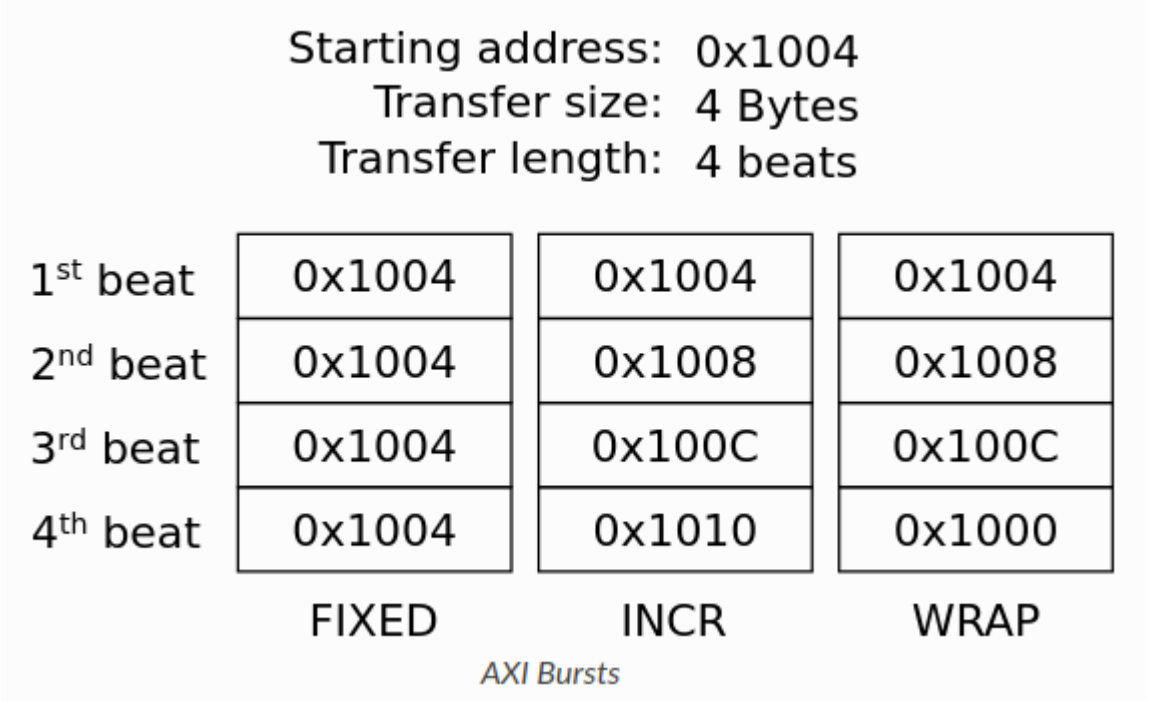
AXI Reads and Writes

AXI4 allows for multiple data transfers over a single request, allowing for greater data bandwidth in the scenario where large amounts of data must be transferred to/from specific addresses. This multi-transfer request is also known as a burst.

All AXI communication is with respect to memory addresses, which each have a specific purpose defined by the RTL and top module.

Three burst types are supported - FIXED, INCR, and WRAP. Each one alters the transfer address in a specific way, allowing for optimal transfers in different situations. For example, a FIXED burst sets every beat to have the same address, which is useful for memory transfers from the same repeated location.

In general, burst addressing specifies where each read or write should be performed in which addresses. Each burst type is as follows

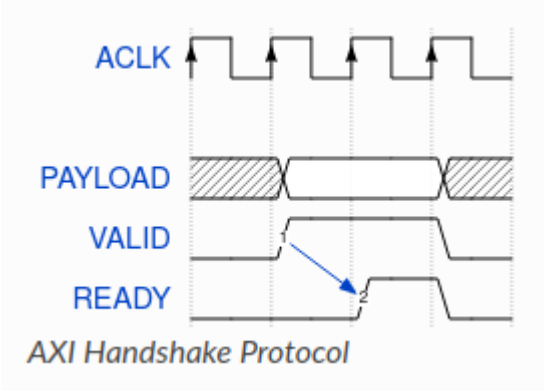


AXI Bursts

AXI4-Lite has no burst protocol (only sending one piece of data at a time) while AXI4-Stream acts as a single unidirectional channel for unlimited data flow between a master and slave, removing the need for addresses.

AXI4 Connections and Channels

In its most basic configuration, the AXI protocol connects and facilitates communication between one master and one slave device. As expected, the master initiates and drives data requests, while the slave responds accordingly. This communication, or transactions as we will now refer to, occurs over multiple channels, each one dedicated to a specific purpose.
The sender must always assert a VALID signal before the receiver and keep it HIGH until the handshake is completed. By using handshakes, the speed and regularity of any data transfer can be controlled.

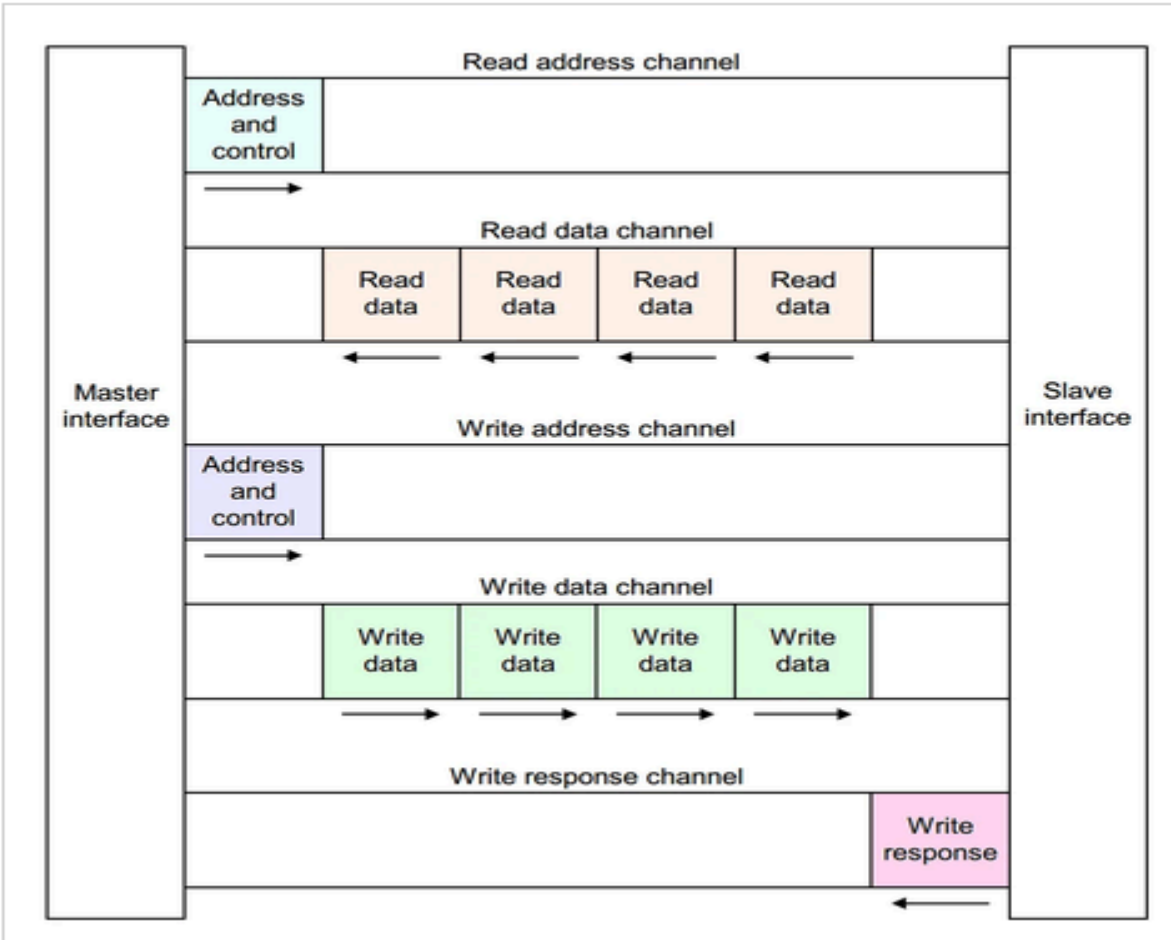


AXI Handshake Protocol

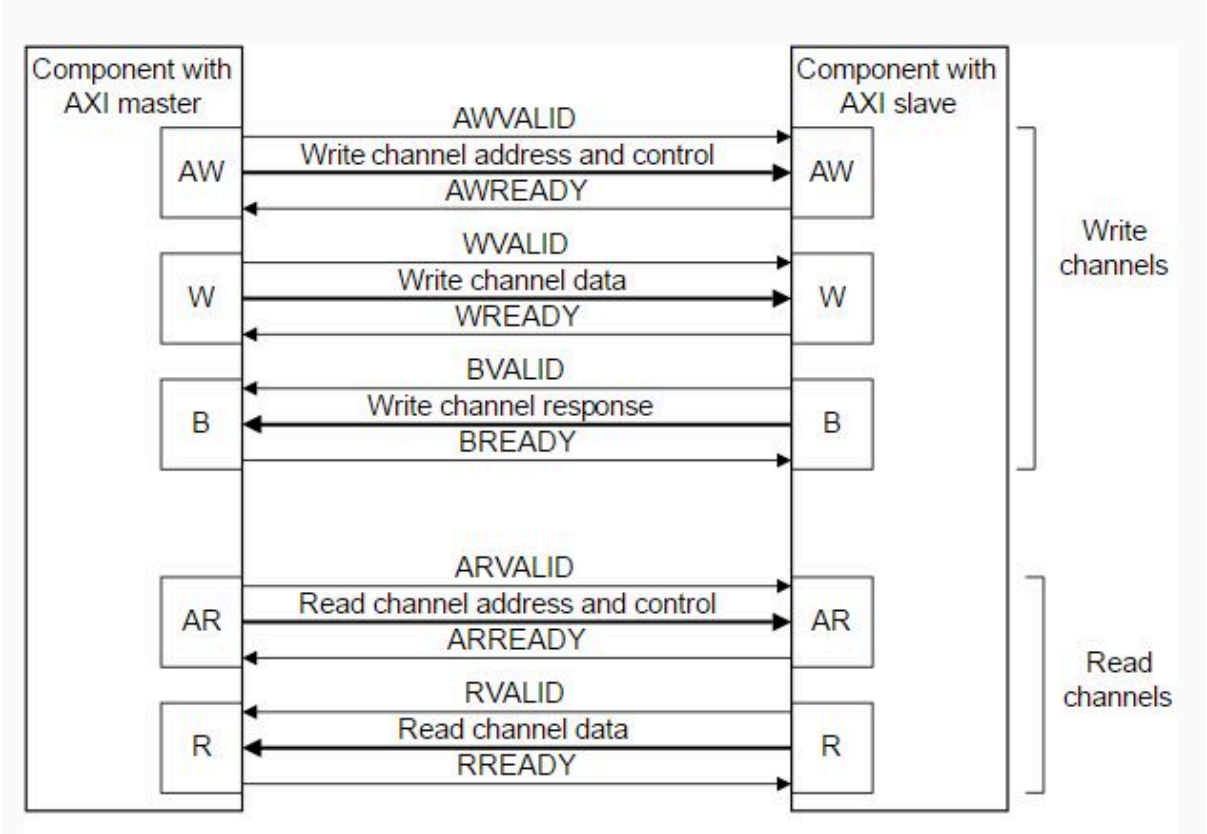
There are five channels, each one transmitting a data payload in one direction. Each channel implements a handshake mechanism, wherein the sender drives a VALID signal

when it has prepared the payload for delivery and the receiver drives a READY signal in response when it is ready to receive the data. The data transfer is also known as a *beat*.

The five AXI4 channels are as follows:



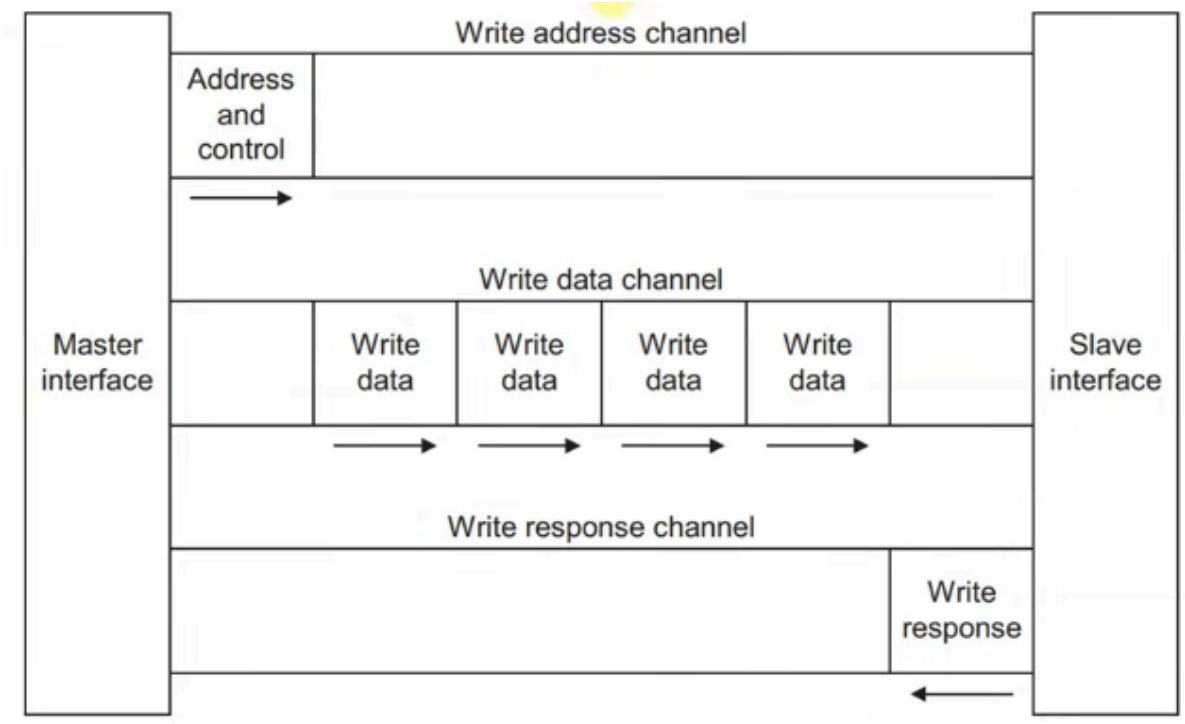
Full AXI Protocol Block Diagram 1

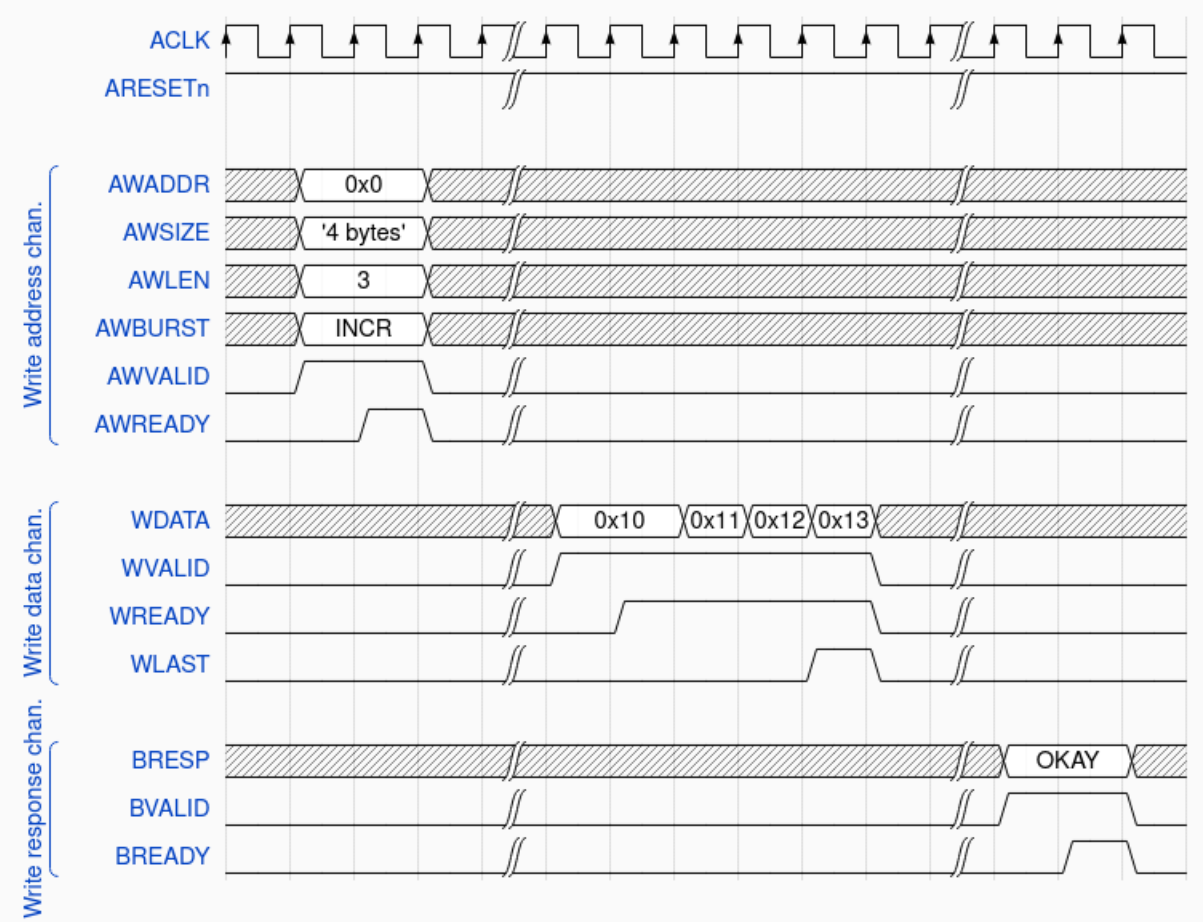


Full AXI Protocol Block Diagram 2

- **Write Address channel (AW):** Provides address where data should be written (`AWADDR`)
  - Can also specify burst size (`AWSIZE`), beats per burst (`AWLEN` + 1), burst type (`AWBURST`), etc.
  - `AWVALID` (Master to Slave) and `AWREADY` (Slave to Master)
- **Write Data channel (W):** The actual data sent (`WDATA`)
  - Can also specify data and beat ID
  - Sender will always assert a finished transfer when done (`WLAST`)
  - `WVALID` (Master to Slave) and `WREADY` (Slave to Master)
- **Write Response channel (B):** Status of write (`BRESP`)
  - `BVALID` (Slave to Master) and `BREADY` (Master to Slave)
- **Read Address channel (AR):** Provides address where data should be read from (`ARADDR`)
  - Can also specify burst size (`ARSIZE`), beats per burst (`ARLEN` + 1), burst type (`ARBURST`), etc.
  - `ARVALID` (Master to Slave) and `ARREADY` (Slave to Master)
- **Read Data channel (R):** The actual data sent back
  - Can also send back status (`RRESP`), data ID, etc.
  - Sender will always assert a finished transfer when done (`RLAST`)
  - `RVALID` (Slave to Master) and `RREADY` (Master to Slave)

Here is an example of a typical read/write AXI transaction.

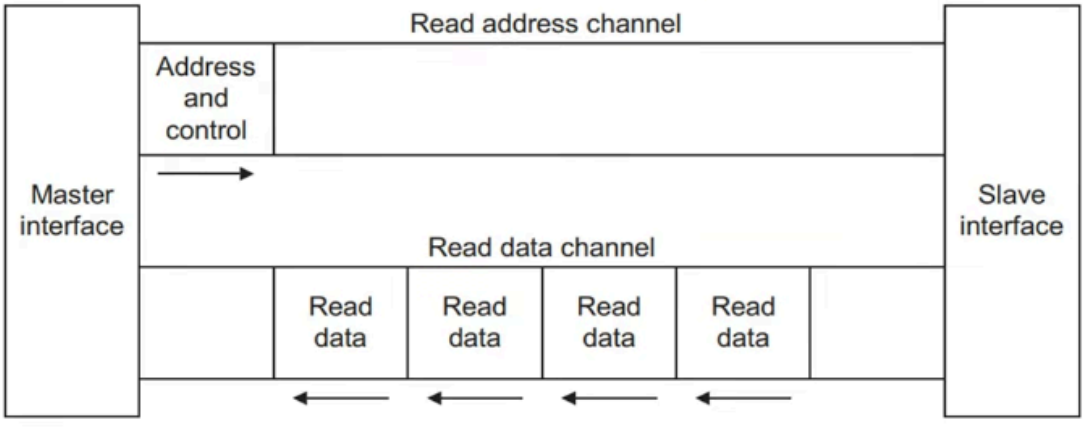
- To write, the master first provides the address (0x0) to write to, as well as the data specifications (4 beats of 4 bytes each, data type of INCR). Both the master and slave then exchange a handshake for verification.
- The master then prepares and writes the actual data payload to send over the channel (0x10, 0x11 0x12, and 0x13), again using a handshake to verify the transfer. The master will signal the end of the payload to the slave using `WLAST`.
- The slave responds with a status of the write and whether it was successful or a failure (all OKAY in this case) and finishes the entire transaction with another handshake
- 

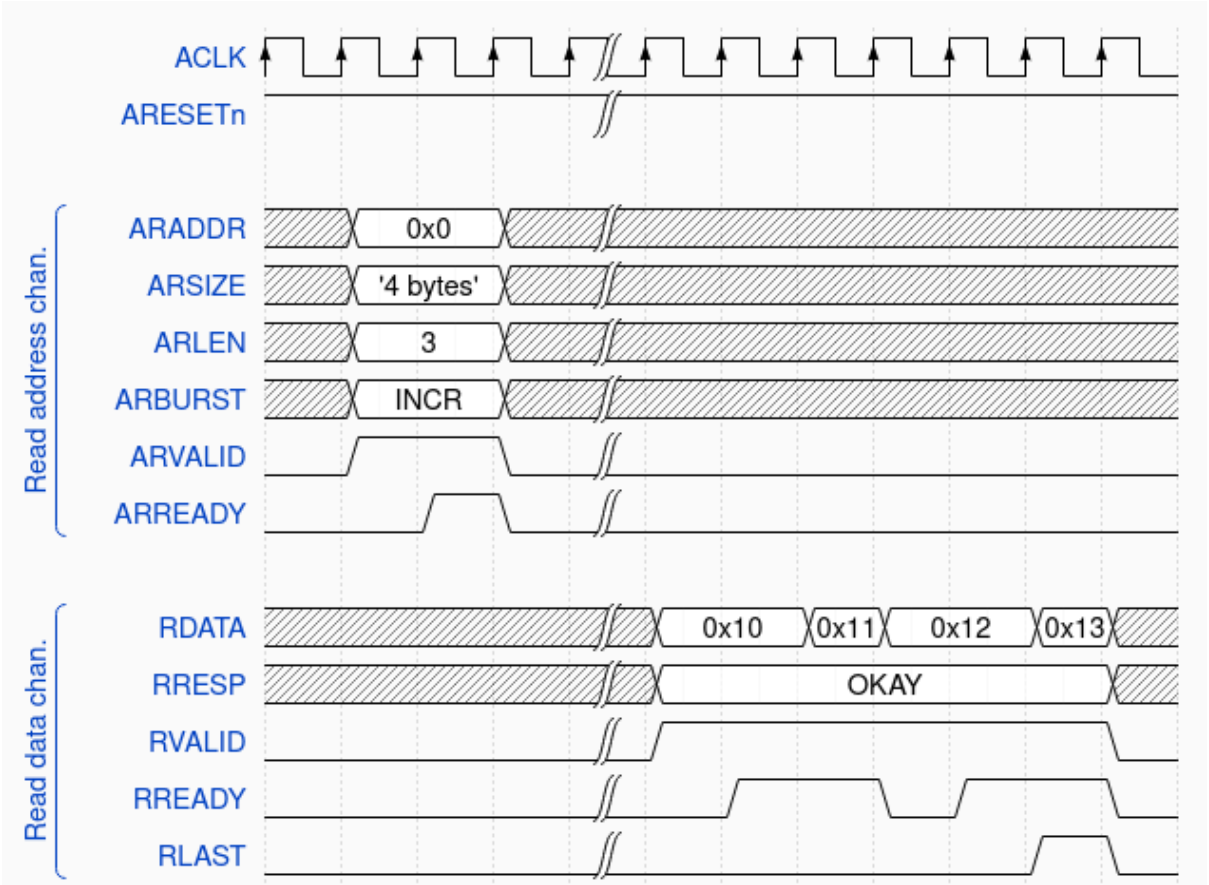


A typical AXI Write transaction

To read, the master first provides the first address to read from (0x0), as well as the data specifications (4 beats of 4 bytes each, data type of INCR). The usual handshake occurs.

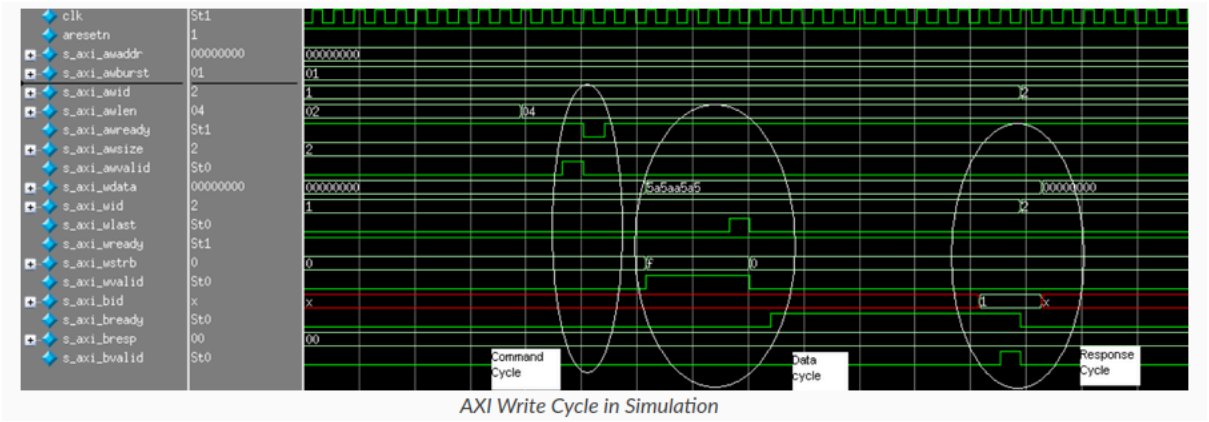
The slave then provides the actual data payload, as well as the status of each beat (all beats are OKAY). The slave will signal the end of the payload to the master using RLAST. As we can see, what was written to the specified addresses was the same as what was read back.





A typical AXI Read transaction

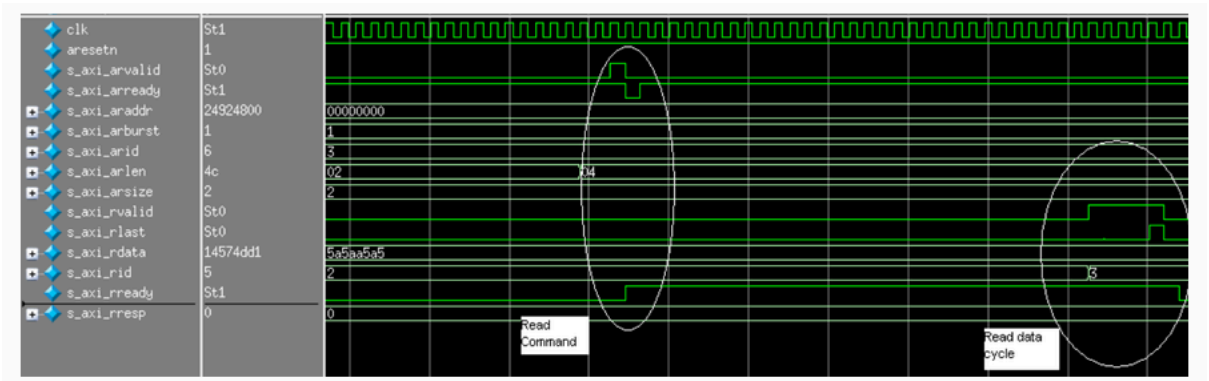
An AXI write consists of a command cycle (define address and burst length), data cycle (putting the data payload over the channel), and a response cycle (checking if the data was received). The master defines the payload specifications and writes the actual data payload (`5a5aa5a5` at address 00000000). The slave toggles `s_axi_bvalid`, exchanging a handshake that signifies the transfer was successful.



AXI Write Cycle in Simulation

Subsequently, an AXI read consists of a read command cycle (again, defining the address to read from, burst length, etc.) and a read data cycle (receiving the data from the requested address). The master specifies the address (00000000) and other

payload specs, receives the data payload from the slave (`5a5aa5a5`), and exchanges a final handshake by toggling `s_axi_rlast` to complete the transfer.



AXI Read Cycle in Simulation

Master Module signal

Signal Name	Type	Width	Description	Channel	Direction	Handshake Signals
ACLK	input	1	Global Clock	Global	Input	N/A
ARESETN	input	1	Active Low Reset	Global	Input	N/A
START_READ	input	1	Start Read Transaction	Control	Input	N/A
START_WRITE	input	1	Start Write Transaction	Control	Input	N/A
address	input	ADDRESS	Target Address	Control	Input	N/A
W_data	input	DATA_WIDTH	Write Data	Control	Input	N/A
ARREADY	input	1	Read Address Channel Ready	Read Address	Input	ARREADY
RDATA	input	DATA_WIDTH	Read Data	Read Data	Input	N/A
RRESP	input	2	Read Response	Read Data	Input	N/A
RVALID	input	1	Read Data Valid	Read Data	Input	RVALID
AWREADY	input	1	Write Address Channel Ready	Write Address	Input	AWREADY
WREADY	input	1	Write Data Channel Ready	Write Data	Input	WREADY
BRESP	input	2	Write Response	Write Response	Input	N/A
BVALID	input	1	Write Response Valid	Write Response	Input	BVALID

ARADDR	output	ADDRESS	Read Address	Read Address	Output	N/A
ARVALID	output	1	Read Address Valid	Read Address	Output	ARVALID
RREADY	output	1	Read Data Ready	Read Data	Output	RREADY
AWADDR	output	ADDRESS	Write Address	Write Address	Output	N/A
AWVALID	output	1	Write Address Valid	Write Address	Output	AWVALID
WDATA	output	DATA_WIDTH	Write Data	Write Data	Output	N/A
WSTRB	output	4	Write Strobes	Write Data	Output	N/A
WVALID	output	1	Write Data Valid	Write Data	Output	WVALID
BREADY	output	1	Write Response Ready	Write Response	Output	BREADY

Slave module signal

Signal Name	**Type**	**Width**	**Description**	**Channel**	**Direction**	**Handshake Signals**
ACLK	input	1	Global Clock	Global	Input	N/A
ARESETN	input	1	Active Low Reset	Global	Input	N/A
ARADDR	input	ADDRESS	Read Address	Read Address	Input	N/A
ARVALID	input	1	Read Address Valid	Read Address	Input	ARVALID

RREADY	input	1	Read Data Ready	Read Data	Input	RREADY
AWADDR	input	ADDRESS	Write Address	Write Address	Input	N/A
AWVALID	input	1	Write Address Valid	Write Address	Input	AWVALID
WDATA	input	DATA_WIDTH	Write Data	Write Data	Input	N/A
WSTRB	input	4	Write Strobes	Write Data	Input	N/A
WVALID	input	1	Write Data Valid	Write Data	Input	WVALID
BREADY	input	1	Write Response Ready	Write Response	Input	BREADY
ARREADY	output	1	Read Address Ready	Read Address	Output	ARREADY
RDATA	output	DATA_WIDTH	Read Data	Read Data	Output	N/A
RRESP	output	2	Read Response	Read Data	Output	N/A
RVALID	output	1	Read Data Valid	Read Data	Output	RVALID
AWREADY	output	1	Write Address Ready	Write Address	Output	AWREADY
WREADY	output	1	Write Data Ready	Write Data	Output	WREADY
BRESP	output	2	Write Response	Write Response	Output	N/A
BVALID	output	1	Write Response Valid	Write Response	Output	BVALID

FSM Logic in Master Module

Current State	**Transition Condition**	**Next State**	**Description**
IDLE	START_READ	RADDR_CHANNEL	Start read operation
IDLE	START_WRITE	WRITE_CHANNEL	Start write operation
RADDR_CHANNEL	ARVALID && ARREADY	RDATA_CHANNEL	Read address handshake complete, move to read data phase
RDATA_CHANNEL	RVALID && RREADY	IDLE	Read data handshake complete, return to idle
WRITE_CHANNEL	AWVALID && AWREADY && WVALID && WREADY	WRESP_CHANNEL	Write address and data handshakes complete, move to response phase
WRESP_CHANNEL	BVALID && BREADY	IDLE	Write response handshake complete, return to idle
default	-	IDLE	Default state for any unspecified conditions

LAB TASK:

TASK 1:

Objective

Design and implement a complete **AXI-Lite interface** from scratch, covering both master and slave components.

Provided Resources

You have been given the following reference materials to support your work:

- **Signal Tables** – Detailed tables listing all input and output signals for both the **Master** and **Slave** modules.
- **FSM Table** – A Finite State Machine (FSM) state transition table specifically for the **Master** module.

Your Task

Using the provided specifications, you are required to:

- Fully develop the **AXI-Lite Master** module.
- Fully develop the **AXI-Lite Slave** module.
- Ensure that your implementation:
    - Accurately reflects all signal definitions from the given tables.
    - Strictly follows the FSM behavior outlined for the master module.
    - Demonstrates correct AXI-Lite protocol operation.

Deliverables

- Complete RTL code Verilog for both master and slave modules.
- Testbench and simulation results(screenshot of wave form) to verify functionality.